



11.2. Binary Search Trees

A tree that knows where to look before it starts searching.

Previous Section:

 [11.1. Operations on Trees](#)

Contents:

[1. Introduction to Binary Search Tree](#)

[2. Traversal on Binary Search Tree](#)

[3. Searching in a Binary Search Tree](#)

[4. Insertion and Deletion in a Binary Search Tree](#)

[5. Implementing a Binary Search Tree in Python](#)

[Summary](#)

[References](#)

So far, we have explored Binary Trees as hierarchical structures that organize data through parent-child relationships. However, while a Binary Tree provides a way to store information, it does not necessarily make searching efficient. This is where the **Binary Search Tree (BST)** comes in.

A Binary Search Tree is one of the most important variations of a Binary Tree because it introduces a simple but powerful rule that allows us to locate values quickly. Instead of placing nodes arbitrarily, a BST arranges them in a way that guides the search process.

You can think of it as a tree that constantly organizes itself so that every comparison eliminates half of the remaining possibilities, much like Binary Search on an array.

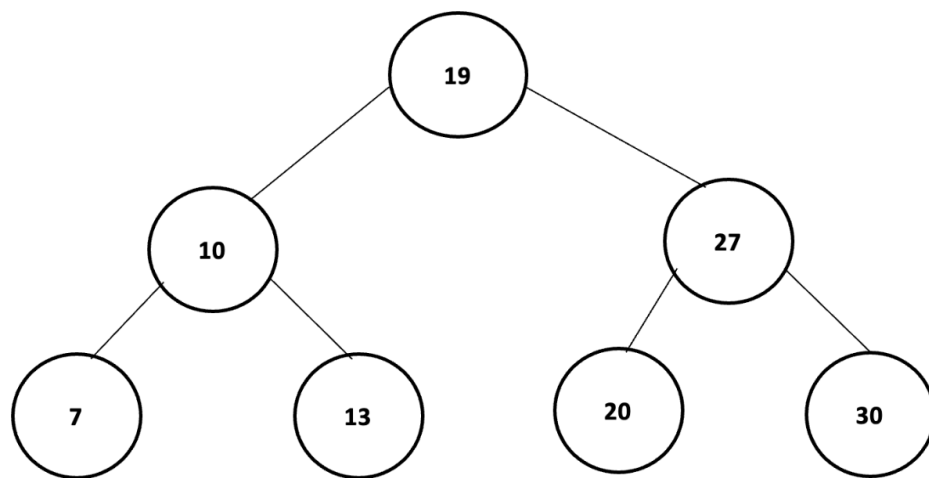
1. Introduction to Binary Search Tree

A **Binary Search Tree (BST)** is a Binary Tree that follows a specific ordering property:

- Every value in the left subtree is smaller than the root node.
- Every value in the right subtree is greater than the root node.

This rule must hold true for every subtree in the tree.

Consider the following BST:



Notice what happens:

- 10 is smaller than 19, so it is placed on the left.
- 27 is greater than 19, so it is placed on the right.
- 7 and 13 are smaller than 19, therefore they belong somewhere inside the left subtree.
- 20 and 30 are greater than 19, therefore they belong somewhere inside the right subtree.

The BST property continues recursively throughout the tree. For example:

- Node 10 has a left child 7 and a right child 13.
- Since $7 < 10 < 13$, the BST property is preserved.
- Node 27 has a left child 20 and a right child 30.
- Since $20 < 27 < 30$, the BST property is preserved.

This ordering gives us an important observation:

- The **leftmost node** usually contains the minimum value.
- The **rightmost node** usually contains the maximum value.

For the tree above:

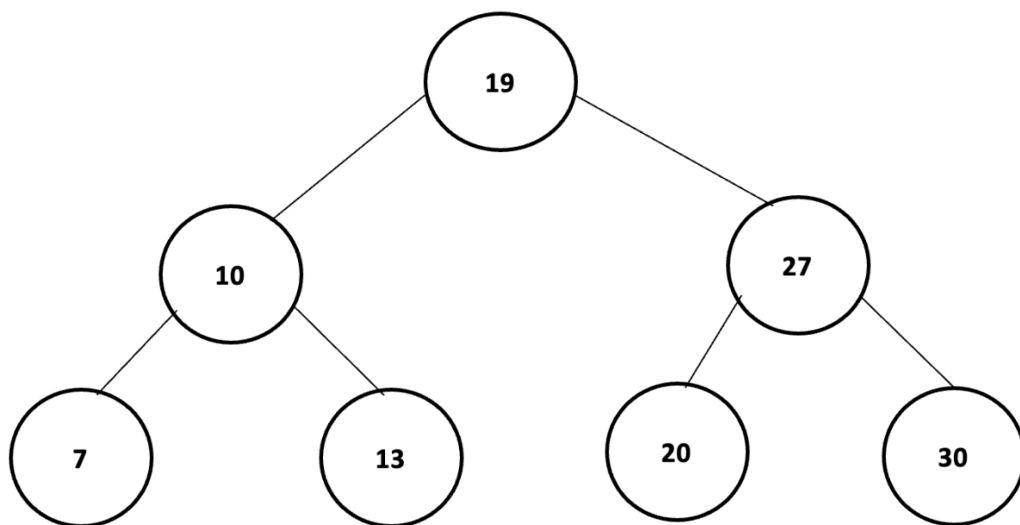
- Minimum Value = 7
- Maximum Value = 30

This property is what makes Binary Search Trees useful for searching, insertion, and deletion operations.

2. Traversal on Binary Search Tree

Just like an ordinary Binary Tree, a Binary Search Tree can be traversed using either Breadth-First Traversal or Depth-First Traversal.

Using the same BST:



- **Breadth-First Traversal:** Visits nodes level by level.

```
[19, 10, 27, 7, 13, 20, 30]
```

We first visit the root, then its children, followed by the next level.

- **In-order Traversal:** The order is *Left* → *Root* → *Right*. Applying In-order Traversal gives:

```
[7, 10, 13, 19, 20, 27, 30]
```

Notice something interesting. The values are automatically sorted in ascending order.

This is one of the most useful properties of a Binary Search Tree. Whenever we perform an In-order Traversal, we obtain the values in sorted order.

- **Post-order Traversal:** The order is *Left* $\rightarrow$ *Right* $\rightarrow$ *Root*

```
[7, 13, 10, 20, 30, 27, 19]
```

The root node is always visited last.

- **Pre-order Traversal:** The order is *Root* $\rightarrow$ *Left* $\rightarrow$ *Right*

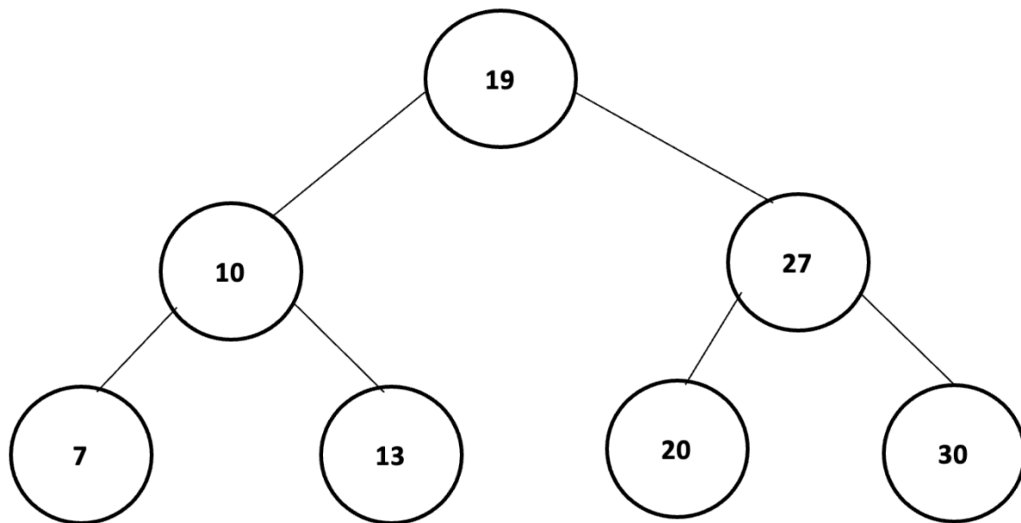
```
[19, 10, 7, 13, 27, 20, 30]
```

The root node is always visited first.

3. Searching in a Binary Search Tree

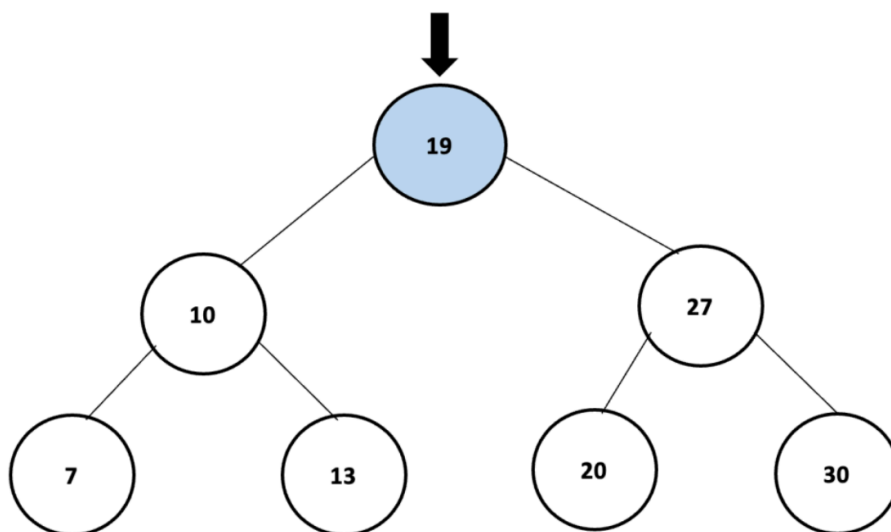
The greatest strength of a BST is its ability to search efficiently. The search process follows the same philosophy as Binary Search on a sorted array.

Suppose we want to find the value **20**. Using the BST below:

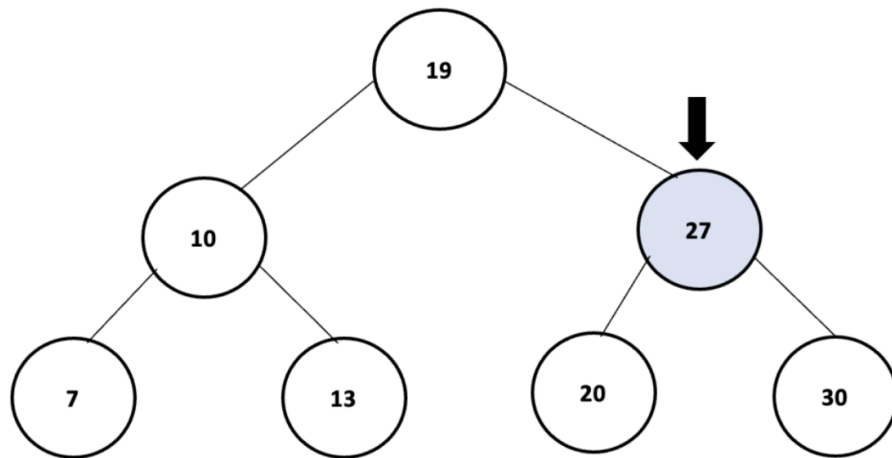


We start at the root node.

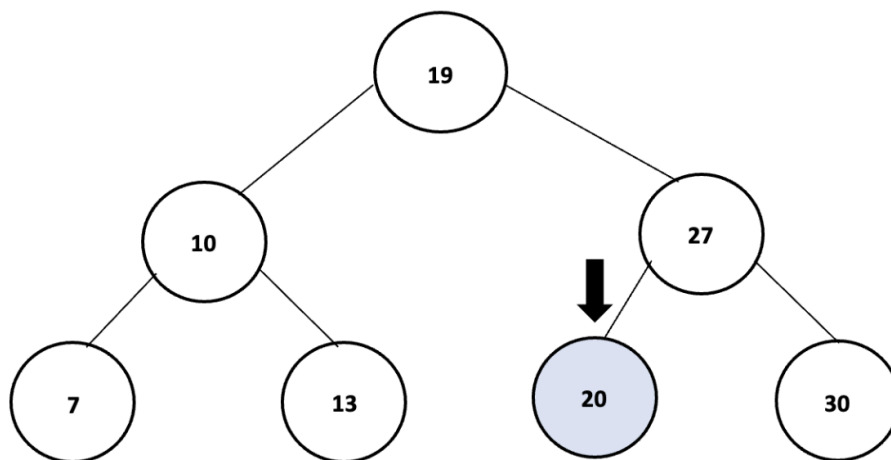
- **Step 1:** Current node = 19 $\rightarrow$ 20 > 19. Since the target value is greater than the current node, we move to the right subtree.



- **Step 2:** Current node = 27 $\rightarrow$ 20 < 27. Since the target value is smaller than the current node, we move to the left subtree.



- **Step 3:** Current node = 20 $\rightarrow$ 20 = 20. The value has been found.



The path taken was: 19 $\rightarrow$ 27 $\rightarrow$ 20. Notice that we never visited: 10, 7, 13, 30. The BST property allowed us to immediately eliminate entire portions of the tree.

Searching Rules

Starting from the root node:

- If Current Value > Target Value $\rightarrow$ Move Left
- If Current Value < Target Value $\rightarrow$ Move Right

- If Current Value = Target Value → Found

These three simple rules are sufficient to search the entire tree.

Time Complexity

- When the tree is balanced: $O(\log n)$. Each comparison removes approximately half of the remaining search space.
- However, if the tree becomes highly unbalanced: $O(n)$. The structure now behaves like a linked list. Searching may require visiting every node, resulting in linear time complexity.

An Alternative Approach

Another possible approach is:

- Perform an In-order Traversal.
- Store the result inside an array.
- Use Binary Search on the array.

For our BST: [7, 10, 13, 19, 20, 27, 30]. Binary Search itself takes: $O(\log n)$. However, generating the sorted array first requires: $O(n)$

Therefore: $O(n) + O(\log n) = O(n)$

Since the traversal dominates the running time, this method is less efficient than directly searching within the BST. The BST was specifically designed to avoid traversing the entire tree, so flattening it into an array defeats its main advantage.

4. Insertion and Deletion in a Binary Search Tree

Searching is not the only operation supported by a Binary Search Tree. In practice, we frequently need to add new values and remove existing ones while preserving the BST property.

The challenge is simple: **Every insertion and deletion must maintain the rule that values in the left subtree are smaller than the parent node, while values in the right subtree are greater.**

Because of this, both insertion and deletion begin with a search process.

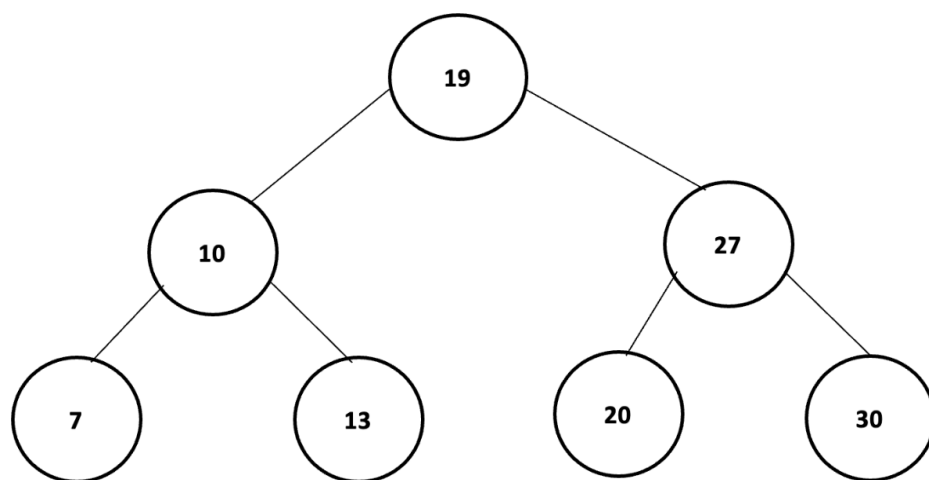
Insertion

When inserting a new value into a BST, the new node is always added as a **leaf node**.

The process is similar to searching:

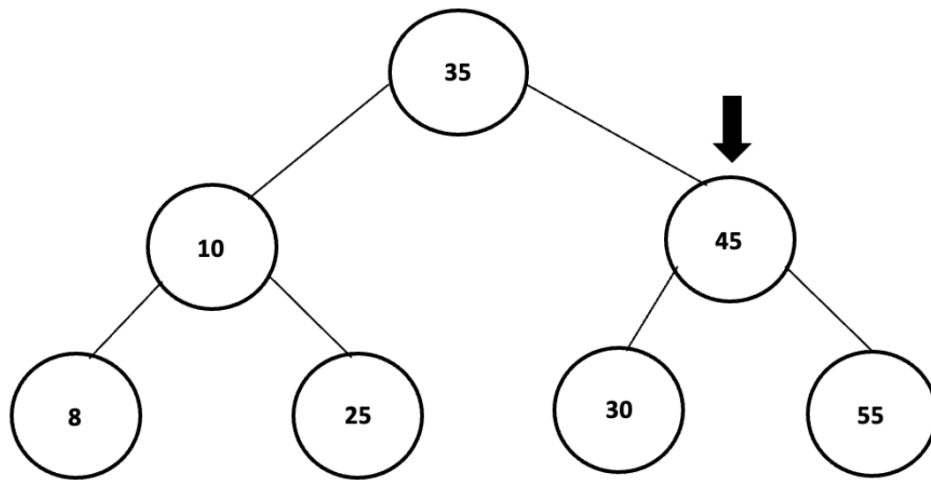
- If the new value is smaller than the current node, move left.
- If the new value is greater than the current node, move right.
- Continue until an empty position is found.

Consider the following BST:

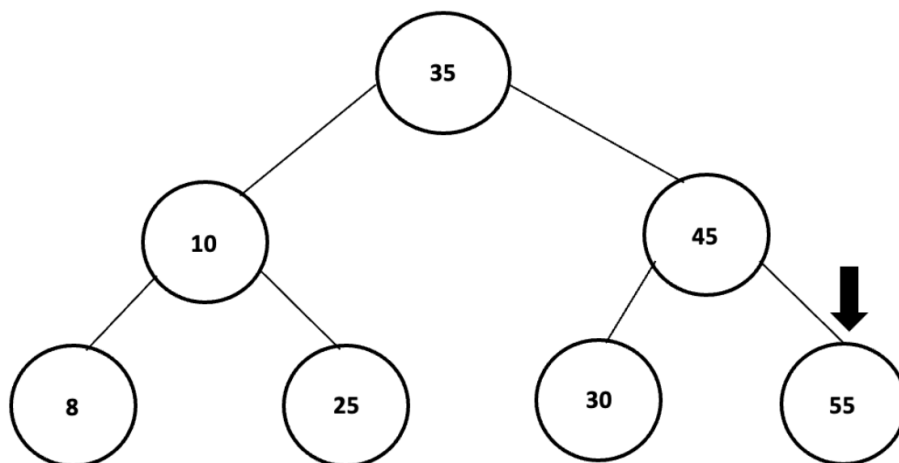


Suppose we want to insert **50**.

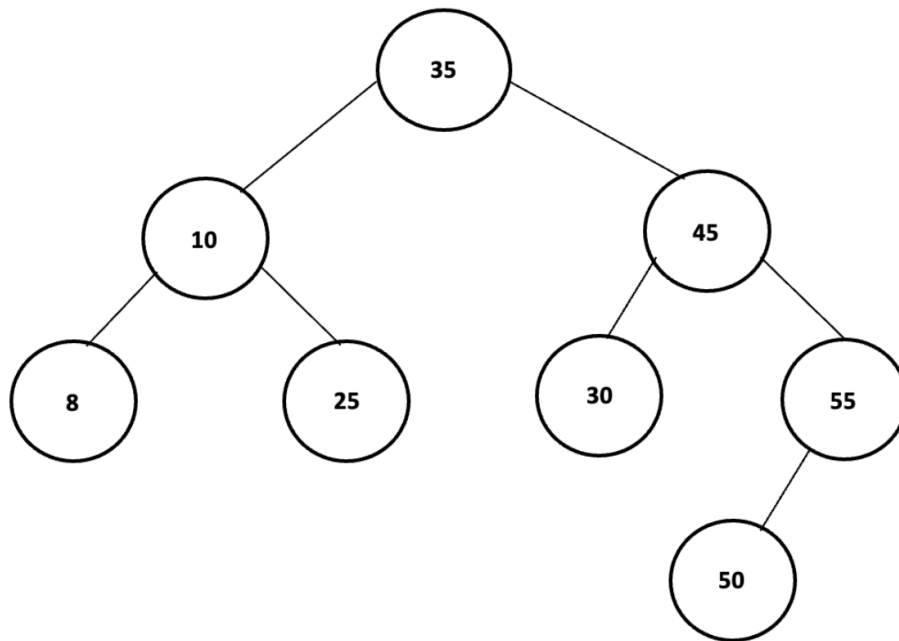
- **Step 1** - Start at the root: $50 > 19$. Move right: $19 \rightarrow 27$



- **Step 2** - $50 > 27$. Move right again: $19 \rightarrow 27 \rightarrow 30$



- **Step 3** - $50 > 30$. Move right. There is no right child. Therefore, insert 50 here.



Notice that the BST property is still maintained. The insertion process follows a single path from the root to a leaf node.

Time Complexity:

- Balanced BST: $O(\log n)$
- Worst-case BST: $O(n)$

Why Insertions Always Occur at Leaf Nodes

Imagine inserting a value directly in the middle of the tree. Doing so would force us to rearrange multiple nodes and potentially violate the BST property.

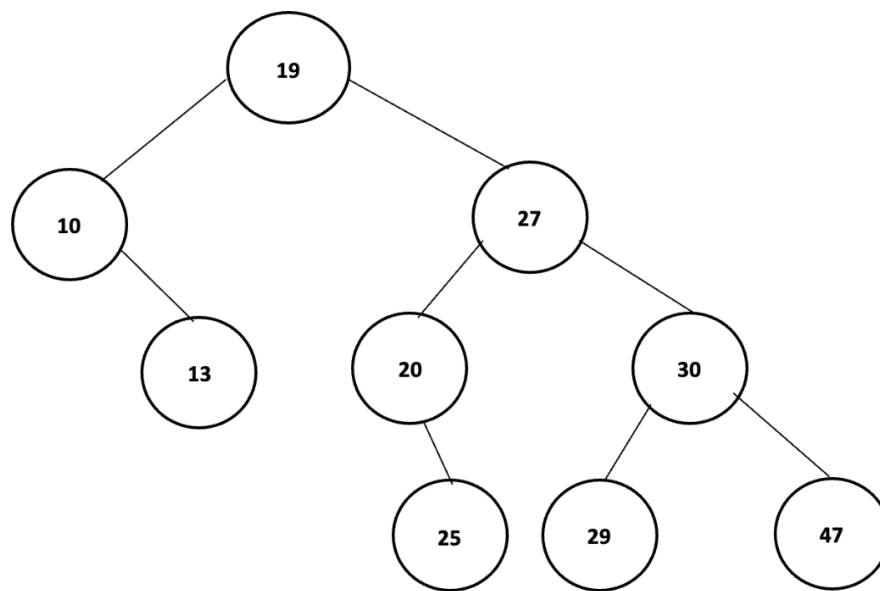
Instead, by always inserting at a leaf position, we guarantee that existing relationships remain valid. The BST property remains intact. The operation stays simple and efficient.

Deletion

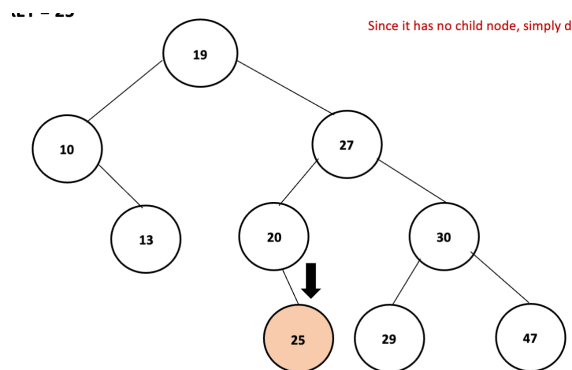
Deletion is more complicated than insertion. After removing a node, we must ensure that the BST remains correctly ordered. There are three possible cases.

- **Case 1 - The Target Node Is a Leaf Node:** This is the easiest case.

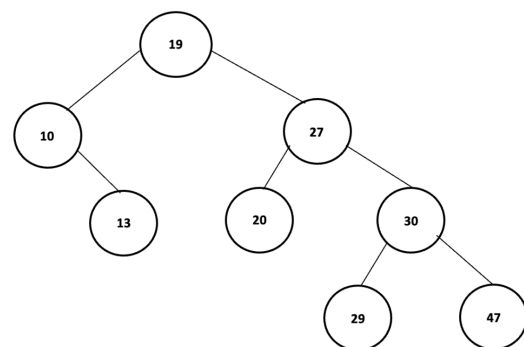
Consider the BST:



Suppose we want to delete **25**. Node 25 has no left child and no right child. Therefore, it is a leaf node. We simply remove it. The resultant tree requires no reorganization.



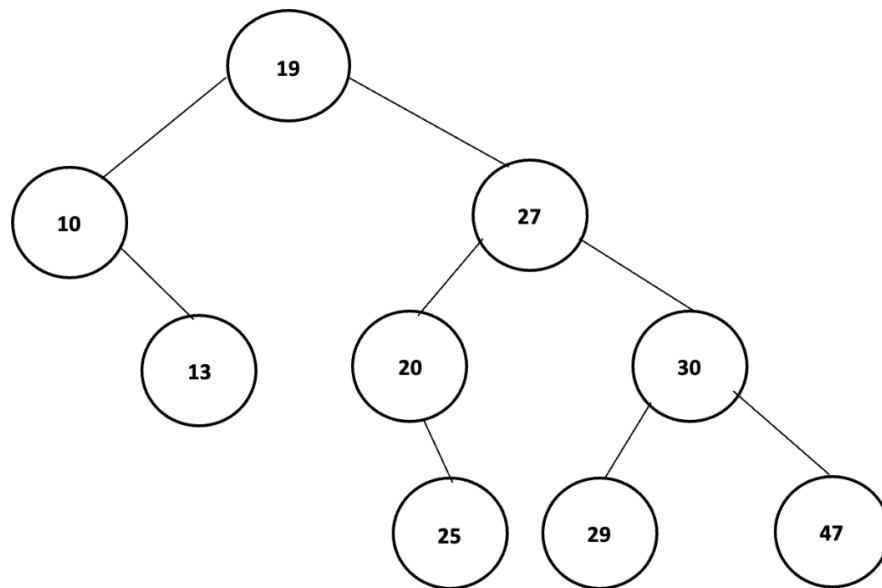
Traverse the tree until reaching leaf node
25



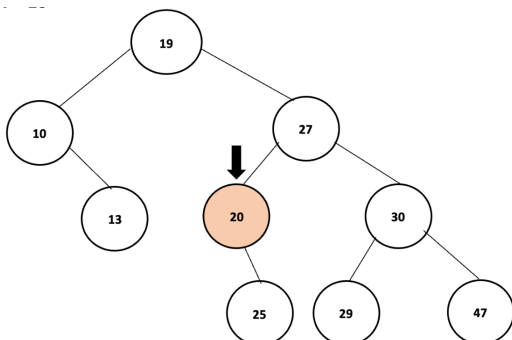
Then simply remove it

- **Case 2 - The Target Node Has One Child**

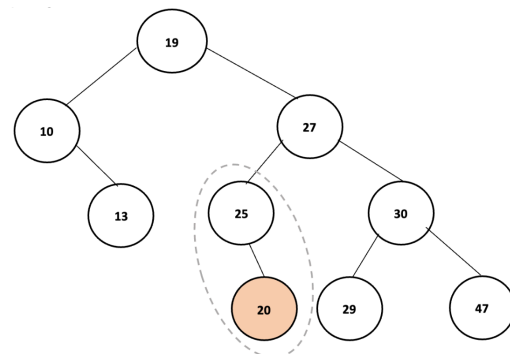
Suppose we have the same tree:



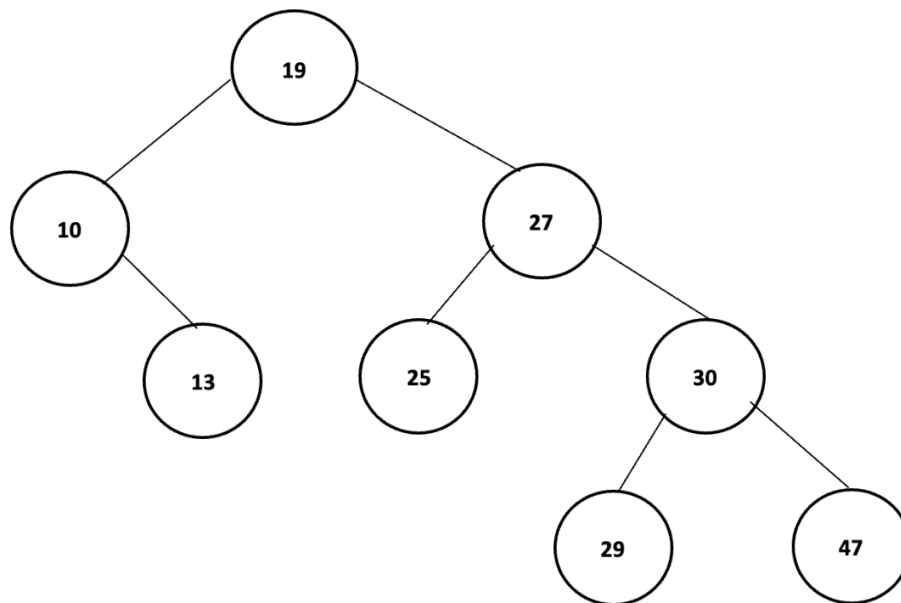
Now we want to delete **20**. Node 20 has one child 25. Instead of removing 20 immediately, we replace it with its child. In the resultant tree, the BST property remains valid because we promote the child to take the place of the deleted node.



Traverse the tree until finding Node 20



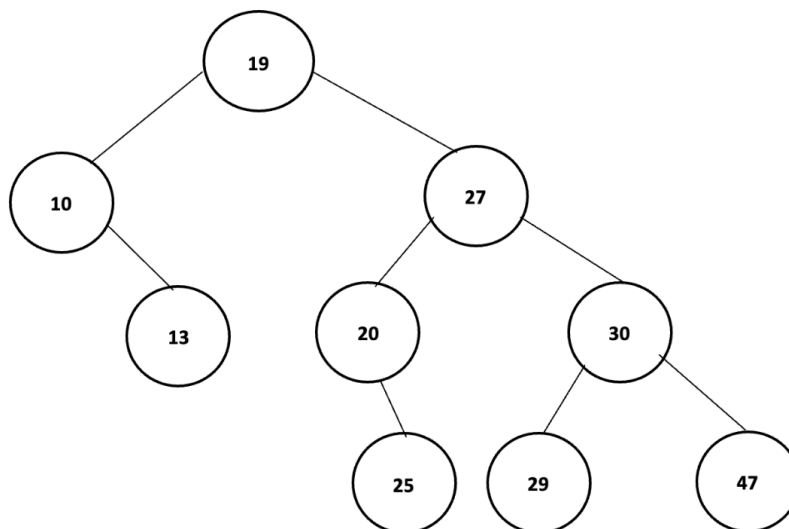
Swap Node 25 and 20



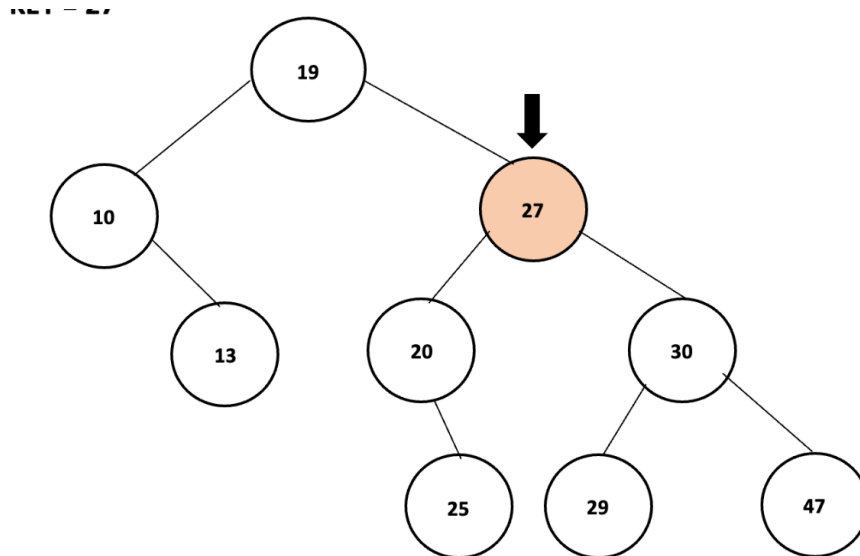
Remove 20 since it is at the leaf node

- **Case 3 - The Target Node Has Two Children:** This is the most interesting case.

Consider the same tree:



Suppose we want to delete **27**. We traverse the tree until we reach the node:



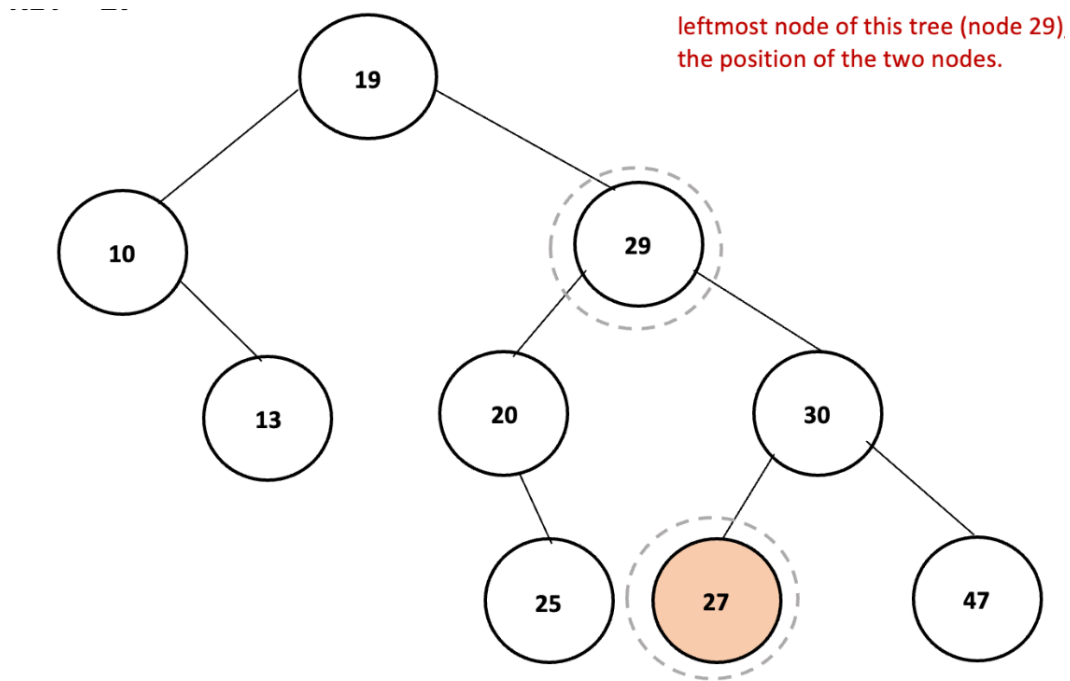
Node 27 has left child 20 and right child 30. Removing it directly would disconnect part of the tree. We need a replacement node.

Using the In-order Successor

A common strategy is:

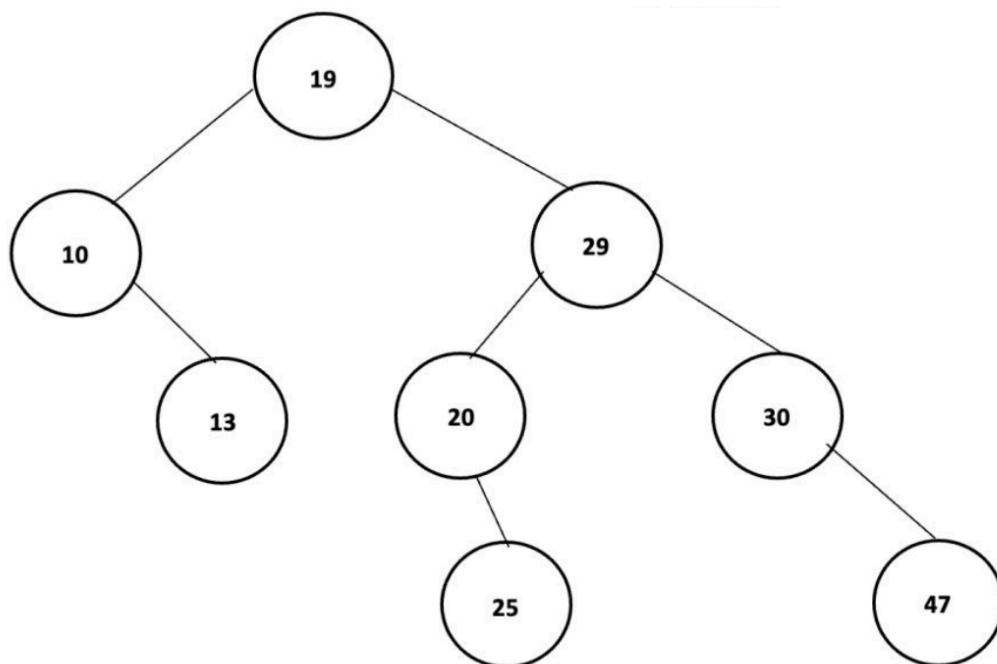
1. Go to the right subtree.
2. Find the leftmost node.
3. Replace the target node with that value.

For node 27:



The leftmost node is: 29 → Therefore: $27 \leftrightarrow 29$

Then we just simply remove 27. After replacement, the BST property is preserved:



The replacement node is called the **In-order Successor** because it appears immediately after the target node during an In-order Traversal.

Using the In-order Predecessor

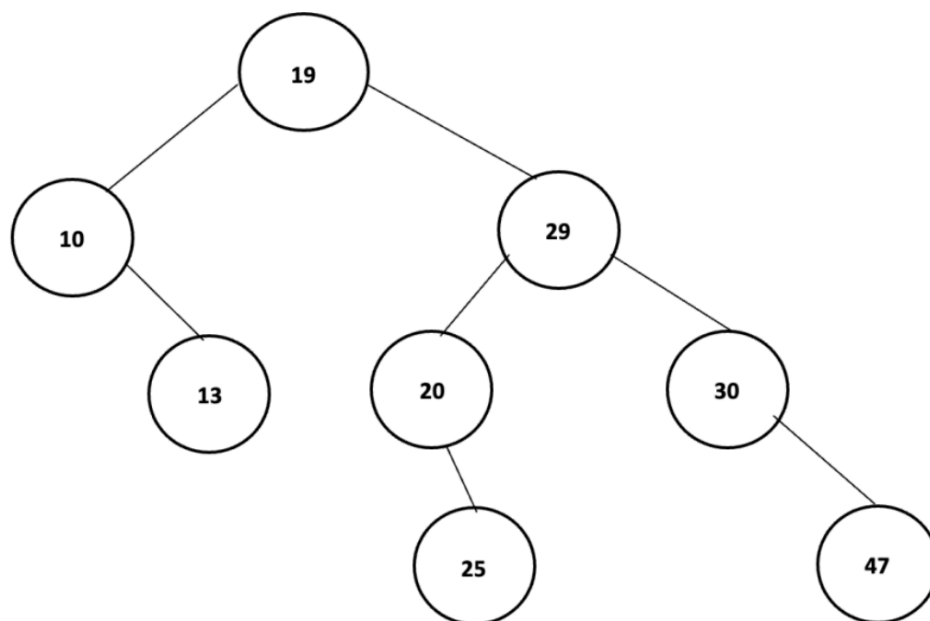
Alternatively, we may use the **In-order Predecessor**. The predecessor is:

- The largest value in the left subtree.
- The value immediately before the target node in an In-order Traversal.

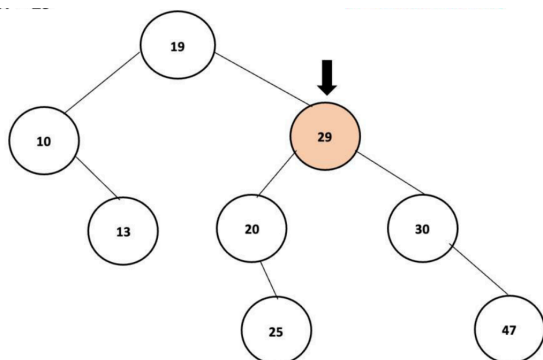
Both methods maintain the BST property.

Why Not Use Any Node?

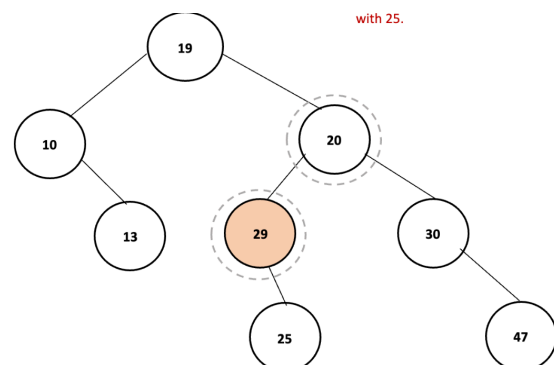
Consider the BST:



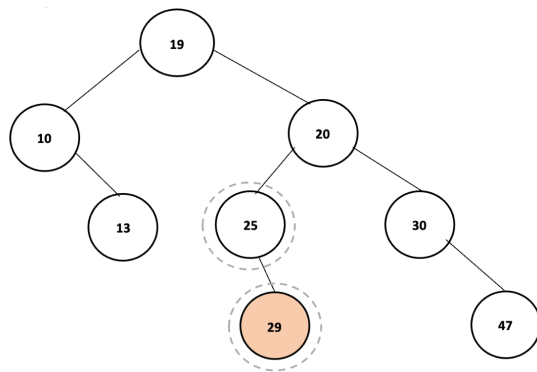
Suppose we delete 29. A tempting idea might be replacing 29 with 20 because 20 is the leftmost node in the subtree.



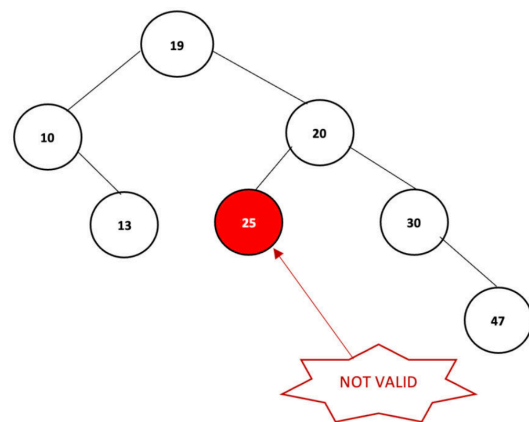
Traverse the tree until we found Node 29



Swap Node 29 and Node 20



Swap Node 29 and Node 25



Delete Node 29 causes violation of BST

Unfortunately, this causes problems. Node 20 still has larger values in its right subtree: **25 and 30**. Moving 20 directly into 29's position may place those values incorrectly, violating the BST property.

For large trees, repeatedly swapping values until the tree becomes valid again would be extremely inefficient. Instead, we choose a value that is guaranteed to fit perfectly. That value is either the In-order Successor or the In-order Predecessor

Finding the Successor Using In-order Traversal

Suppose the In-order Traversal of a BST is:

```
[10, 13, 19, 20, 25, 29, 30, 47]
```

For target node 29: **Successor = 30** and **Predecessor = 25**. Either value can replace 29 safely.

This is why successor and predecessor methods are widely used in BST deletion algorithms. They guarantee that the Binary Search Tree remains ordered after the deletion operation.

Time Complexity

Both insertion and deletion involve searching through the tree. Therefore:

- Balanced BST: $O(\log n)$
- Worst-case BST: $O(n)$

Deletion with two child nodes requires an additional search for the successor or predecessor, but this search follows only a single path through the tree and does not change the overall complexity.

For this reason, successor and predecessor replacement remains the standard approach for deleting nodes with two children in a Binary Search Tree.

5. Implementing a Binary Search Tree in Python

So far, we have studied the theory behind Binary Search Trees, including traversal, searching, insertion, and deletion. Now it is time to implement these operations in Python.

Our implementation uses two classes:

- `Node` represents a single node in the tree.
- `BinarySearchTree` manages the entire BST and its operations.

The implementation relies heavily on recursion because every subtree in a BST is itself a smaller BST.

The Node Class

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

    def display(self):
        if self.data is not None:
            print(self.data, end=" ")
```

Every BST is composed of nodes. Each node stores:

- A value (`data`)
- A reference to the left child (`left`)
- A reference to the right child (`right`)

Initially, both child references are set to `None` because a newly created node has no children.

The `display()` method is a helper function used during traversal. It simply prints the value stored in the node.

Processing a Node

```
def process_node(node):  
    if node is not None:  
        node.display()
```

This helper function is responsible for processing a node during traversal.

Currently, processing means displaying the node value. Separating the processing logic from the traversal logic makes the code easier to modify later. For example, instead of printing values, we could store them inside a list or perform calculations.

Initializing the Binary Search Tree

```
class BinarySearchTree:  
    def __init__(self):  
        self.root = None  
        self.level = 0
```

The constructor creates an empty BST. Initially:

- `root` is `None` because no nodes exist yet.
- `level` is initialized to zero.

The root node acts as the entry point for all BST operations.

Traversing the Tree

```
def traverse(self):  
    self.inorder_traverse(self.root)
```

This function starts an In-order Traversal from the root node. Instead of directly traversing the tree, it delegates the work to the recursive traversal function.

Recursive In-order Traversal

```
def inorder_traverse(self, root_node):  
    if root_node is None:  
        return  
  
    self.inorder_traverse(root_node.left)  
    process_node(root_node)  
    self.inorder_traverse(root_node.right)
```

```
    19  
   / \  
  10  27  
 /\  
7 13 20 30
```

Produces: 7 10 13 19 20 27 30

This function implements the In-order Traversal algorithm. The recursion works as follows:

1. Traverse the left subtree.
2. Visit the current node.
3. Traverse the right subtree.

Because BSTs store smaller values on the left and larger values on the right, an In-order Traversal automatically prints the values in ascending order.

Inserting a Node

```
def insert(self, data):  
    self.root = self.insert_tree(self.root, data)
```

This function acts as the public insertion method. It starts the insertion process from the root node.

Recursive Insertion

```
def insert_tree(self, current, data):
    if current is None:
        return Node(data)

    if data < current.data:
        current.left = self.insert_tree(current.left, data)

    elif data > current.data:
        current.right = self.insert_tree(current.right, data)

    else:
        print(f"Node {data} already exists")
        return current

    return current
```

This function recursively searches for the correct position for the new value. There are three possibilities:

- **Case 1: Empty Position Found**

```
if current is None:
    return Node(data)
```

The insertion point has been found. A new node is created and returned.

- **Case 2: Value Is Smaller** `if data < current.data:`

Move to the left subtree. Since all smaller values belong on the left side, the insertion process continues there.

- **Case 3: Value Is Larger** `elif data > current.data:`

Move to the right subtree. Since all larger values belong on the right side, the insertion process continues there.

- **Duplicate Value:** The value already exists. No insertion occurs because duplicate values are not allowed in this implementation.

Finding the Minimum Value

```
def find_min_tree(self, root):  
    if root.left is None:  
        return root.data  
    else:  
        return self.find_min_tree(root.left)
```

```
    27  
    /  
   20  
  /  
 15  
 /  
10
```

The minimum value is: 10

This function finds the smallest value in a subtree.

Recall an important BST property: The leftmost node always contains the minimum value. The algorithm repeatedly moves left until no further left child exists.

This is particularly useful during deletion when we need to locate the In-order Successor.

Deleting a Node

```
def delete(self, data):  
    self.root = self.delete_tree(self.root, data)
```

This function starts the deletion process from the root node. The actual work is performed by the recursive deletion function.

Recursive Deletion

```
def delete_tree(self, current, data):  
    if current is None:
```

```

        print(f"Node {data} not found")
        return None

    if data == current.data:
        if current.left is None and current.right is None:
            return None

        if current.left is None:
            return current.right

        if current.right is None:
            return current.left

        min_right = self.find_min_tree(current.right)
        current.data = min_right
        current.right = self.delete_tree(current.right,
min_right)
        return current

    if data < current.data:
        current.left = self.delete_tree(current.left, d
ata)
        return current
    else:
        current.right = self.delete_tree(current.right,
data)
        return current

```

Deletion is the most complex BST operation because removing a node may break the BST structure. There are three major cases.

- **Case 1: Node Not Found**

```

if current is None:
    print(f"Node {data} not found")

```

The search reached an empty branch. The value does not exist in the tree.

- **Case 2: Target Node Found** `if data == current.data:`

Once the target node is found, we determine which deletion scenario applies.

Scenario A: Leaf Node

```
if (current.left is None) and (current.right is None):  
    return None
```

The node has no children. It can simply be removed.

Scenario B: One Child

```
if current.left is None:  
    return current.right
```

```
if current.right is None:  
    return current.left
```

The node has exactly one child. We replace the node with its child.

Scenario C: Two Children

```
min_right = self.find_min_tree(current.right)  
current.data = min_right  
current.right = self.delete_tree(current.right, min_right)
```

This is the most challenging case. The process is:

1. Find the minimum value in the right subtree.
2. Replace the target value with that value.
3. Delete the duplicate node from the right subtree.

The minimum value in the right subtree is the In-order Successor.

- **Continue Searching** - If the target node has not been found:

```
if data < current.data:  
    current.left = self.delete_tree(current.left, data)  
    return current  
else:  
    current.right = self.delete_tree(current.right, data)  
    return current
```

The algorithm continues searching either the left or right subtree according to the BST rules.

Complete Binary Search Tree Implementation

[bst.py](#)

Binary Search Tree Visualization

<https://www.cs.usfca.edu/~galles/visualization/BST.html>

Summary

In this section, we implemented a complete Binary Search Tree in Python. We created the `Node` class to represent individual tree nodes and the `BinarySearchTree` class to manage the overall structure. Using recursion, we implemented In-order Traversal, insertion, minimum-value searching, and deletion while maintaining the BST property.

The BST significantly improves searching efficiency compared to a regular Binary Tree because each comparison allows us to eliminate half of the remaining search space. However, this advantage depends on the tree remaining reasonably balanced. When a BST becomes heavily skewed, its performance degrades from $O(\log n)$ to $O(n)$, making it behave more like a linked list than a tree.

This limitation motivates the next topic: **AVL Trees**. AVL Trees are self-balancing Binary Search Trees that automatically reorganize themselves after insertions and deletions. By maintaining balance, they ensure that searching, insertion, and deletion remain efficient, preserving the desirable **$O(\log n)$** performance even as the tree grows.

References

https://drive.google.com/file/d/1tOqiD1hwn4q_ZbZHeOp-e3oeJfgTEdDq/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/binary-search-tree-data-structure/>

<https://www.youtube.com/watch?v=Gt2yBZAhsGM>

Next Section:

11.3. AVL Trees